

AIRBNB LISTING EXPERIENCE CLONE

PlayPower Labs — Full-Stack Take-Home Assignment

Engineering Plan & Technical Design Document

Listing Page · Photo Tour · Lightbox · AI Travel Concierge · MERN Backend Foundation

Prepared by: **Rushi Hansora**

B.Tech Information Technology, Dharmsinh Desai University

GitHub: Rushi-Hansora | work.rushihansora111@gmail.com

Document type: Pre-implementation plan (AI-native development workflow)

1. Executive Summary

This document is the engineering plan for the PlayPower Labs take-home assignment: an original, pixel-conscious clone of an Airbnb-style listing experience. The reference implementation (*airbnb-clone-umber-two.vercel.app*) is used strictly as a visual and behavioral specification — no source code, assets, or markup are copied. The build follows a priority-ordered scope so that the required assignment is never put at risk by nice-to-have additions.

The approach demonstrates full-stack ownership across five disciplines: frontend architecture (React + Vite + Tailwind), backend architecture (Node/Express/MongoDB with a layered service/repository pattern), testing (unit, integration, Playwright E2E), DevOps (GitHub Actions CI/CD), and an AI-native development workflow (specialized agent roles, documented prompt sequence). A single differentiating feature — an AI Travel Concierge — is layered on only after the core clone reaches parity.

Priority Framework

Tier	Focus	Rule
P0	Required assignment — Listing page, Photo Tour, Lightbox	Never delayed or degraded by anything below it
P1	Engineering quality — clean React, layered backend, tests	Built alongside P0, not after it
P2	Differentiator — AI Travel Concierge prototype	Starts only once P0 is visually and behaviorally complete
P3	Backend / DevOps foundation	Clean but never a blocker for the frontend deliverable
P4	Future extensibility	Documented as extension points, not built now

2. Core Assignment Scope

2.1 Listing Page

A full recreation of the listing experience: header/navigation, search bar, title and share/save controls, image hero, property and host information, amenities, sleeping arrangements, reviews, location and neighborhood details, house rules, cancellation policy, safety information, nearby listings, and a sticky reservation card with pricing, date pickers, guest selector, and CTA. Visual QA is tracked against the reference for spacing, typography, radii, shadows, iconography, image ratios, hover/transition behavior, and accessibility of every interactive element.

2.2 Photo Tour

Triggered from "Show all photos," the hero images, or gallery entry points. Implements a categorized grid, lazy-loaded thumbnails, keyboard accessibility, proper focus handling on open/close, and smooth scroll and transition behavior.

2.3 Lightbox

Single-image view reachable from any gallery thumbnail: previous/next controls, close button, image counter, ← / → / Esc keyboard support, focus trapping, accessible labels, boundary handling at the first and last image, and

animated transitions between images.

Desktop-first per the assignment brief (mobile explicitly out of scope), but components and CSS are written with grid/flexbox/clamp() so responsive layouts can be layered in later without rework.

3. Technology Stack

Layer	Technologies
Frontend	React, Vite, React Router, Tailwind CSS, Motion for React, Lucide React, Axios, TanStack Query, Context API
Backend	Node.js, Express.js, MongoDB, Mongoose, JWT auth, REST APIs (repository → service → controller → middleware → validation layers)
DevOps	GitHub Actions CI, ESLint/Prettier, Vitest, Playwright, dependency auditing, deployment-ready config
Deployment	Frontend prototype on Vercel; backend architected and deployment-ready, decoupled so it never blocks the clone

Exact dependency versions are pinned from official documentation before implementation begins and recorded in *docs/VERSIONS.md* (package, version, rationale, official source, notes) — no version is assumed.

Architectural clarification

TanStack Query is the **server-state** layer (fetching, caching, invalidation, mutations) — it is not used as an SSR framework. Context API is reserved for lightweight client/UI state (selected gallery image, modal state, booking UI state). This separation is deliberate and documented so it doesn't blur under time pressure.

4. Frontend Architecture

A feature-oriented structure keeps UI, business logic, and data access separate: *app/* → *components/* → *features/* → *hooks/* → *contexts/* → *services/* → *utils/* → *pages/*. The data-flow convention is enforced throughout:

- Client state: **Component** → **custom hook** → **Context**
- Server state: **Component** → **TanStack Query hook** → **API service** → **Axios** → **backend**

React practices demonstrated: functional components, custom hooks, controlled components, derived rather than duplicated state, justified use of *useMemo/useCallback*, error boundaries, route-level code splitting with *React.lazy/Suspense*, and a clean separation between UI and service logic. Patterns used only where they earn their keep — Compound Components (gallery/lightbox), Provider pattern (booking/session UI state), Adapter pattern (API response transformation) — are each documented with what problem they solve in *docs/REACT_PATTERNS.md*.

5. Backend Architecture

Even though the assignment marks the backend optional, a clean foundation is built to demonstrate full-stack ability, following a strict layering: *Route* → *Controller* → *Service* → *Repository* → *Mongoose Model* → *MongoDB*, backed by middleware, validation, centralized error handling, structured logging, and configuration management. SOLID principles are applied where they add real clarity, not forced into every file.

Concern	Approach
Auth	JWT-based; access token in secure HttpOnly cookie; password hashing; expiration/refresh strategy documented
Errors	Centralized Express middleware; typed errors (AppError, ValidationError, AuthenticationError, NotFoundError); no stack traces leaked in production
API design	REST under /api/v1, consistent response envelopes, pagination (page/limit/total/hasNextPage)
Search	Simple Mongoose queries for the prototype; documented evolution path to Atlas Search / Elasticsearch
Observability	Request count, latency, error count, booking/concierge usage hooks; lightweight /api/v1/metrics
Security	Helmet, CORS policy, rate limiting, input sanitization, secrets via env vars, dependency audits

6. Differentiator — AI Travel Concierge

Added only after P0 parity is reached, and clearly labeled as a project differentiator rather than an official Airbnb feature. It is built as an agent-style pipeline, not a scripted chat window, so a real LLM can later replace the deterministic ranking core without touching the interfaces:

User Request → Intent Parser → Constraint Extractor → Property Search → Filtering → Ranking → Recommendation Generator → User Selection → Booking Flow

Given a natural-language request (destination, budget, minimum rating, radius, property type, dates), the service parses constraints, ranks the mock property dataset, and returns each recommendation with a rating, estimated price, distance, amenities, and an explicit match score plus reasoning. The user can either view a property or continue to a booking summary; payment is a safe, mock/test-only flow — no real transactions. Interfaces (*AIConciergeService*, *PropertySearchService*, *PropertyRankingService*, *BookingService*) are deterministic and unit-testable today.

7. Engineering Practices

Accessibility

Keyboard navigation, visible focus states, ARIA (label/expanded/controls), dialog semantics with focus trapping, Escape handling, alt text, prefers-reduced-motion support.

Performance

Route-level code splitting, image lazy loading, skeleton states, TanStack Query caching, Vite production optimization, pagination; virtualization only if genuinely needed.

Testing

Unit tests (pricing, filters, ranking, constraint extraction, pagination); integration tests (API endpoints, auth, repository/service interactions); Playwright E2E across the listing → gallery → lightbox → booking → concierge flow; a documented smoke-test checklist.

Security

Helmet, CORS, rate limiting, schema validation on every mutating endpoint, secure cookie flags, no secrets in Git, dependency auditing in CI.

SEO

Semantic HTML, meta/Open Graph tags, canonical URLs, meaningful heading hierarchy — kept lightweight so it never competes with visual fidelity for time.

DevOps

GitHub Actions for frontend and backend: install → lint → test → build → audit → deploy, with secrets handled via environment variables, never hard-coded.

8. AI-Native Development Workflow

The build itself is run as a multi-agent workflow rather than one undifferentiated pass, so each stage has a clear owner and a clear definition of done:

Planning → UI Analysis → Frontend Implementation → Accessibility Review → Testing → Performance Review → Backend → Security Review → DevOps Review → Final QA

Agent and skill configuration lives in `.agents/` (frontend, backend, testing, ui-reviewer, security-reviewer, devops) with project-wide rules captured in `CLAUDE.md`. Every prompt used during development is logged in `docs/AI_PROMPT_SEQUENCE.md` so the process is auditable — the point is to show deliberate use of AI tooling, not to hide it.

9. Delivery Roadmap

Sprint	Deliverable
Sprint 0	Requirements, reference analysis, architecture, version pinning
Sprint 1	Listing page — full visual and structural parity
Sprint 2	Photo Tour and Lightbox, including keyboard/focus behavior
Sprint 3	Visual QA pass and accessibility hardening
Sprint 4	AI Travel Concierge prototype
Sprint 5	Backend foundation (auth, listings, bookings, concierge API)
Sprint 6	Unit, integration, and Playwright E2E tests; CI/CD pipelines
Sprint 7	Final QA, documentation pass, deployment

10. Originality & Documentation

The reference site is observed for behavior only — no source code, markup, styles, or copyrighted images are copied or scraped; property imagery is original or generated and attributed. The finished submission ships with a full documentation set: requirements, reference analysis, architecture (with a diagram), tech decisions, versions, SDLC, React patterns, storage strategy, SEO, accessibility, security, observability, DevOps, testing, smoke-test checklist, AI concierge design, AI workflow, visual QA notes, and the prompt sequence — alongside the working application, its test suite, and CI configuration.

Guiding principle: a clean, complete implementation beats an over-engineered incomplete one. Every addition is tested against one question — “does this improve the assignment score?” — before it is built.